

Hotel Recommendation System – RAG Pipeline

Milestone 3: Performance Evidence

Authors	Emmanuel Mukange (ZDA24B029) Raudhat Ramadhan (ZDA24B010)
Course	CS6040 – Database Management Systems, IIT Madras Zanzibar
Track	A – RAG Pipeline (Retrieval-Augmented Generation)
Due	5 June 2026, 23:59 EAT

Project Summary

The Hotel Recommendation System is a Retrieval-Augmented Generation (RAG) pipeline designed to answer natural-language traveller queries such as *“Find me a well-reviewed budget hotel in Nairobi.”* The system combines a normalised PostgreSQL relational database with a Pinecone vector store. When a user submits a query, sentence-transformer embeddings retrieve semantically relevant hotel records from Pinecone; those candidate hotel IDs are then passed to PostgreSQL, where relational aggregations over reviews, bookings, and star ratings produce the final ranked response. The relational schema comprises five tables in Third Normal Form: **room_types**, **hotels**, **users**, **bookings**, and **reviews**. The dataset contains 1,553 rows in total, combining real hotel records sourced from Kaggle with synthetically generated users, bookings, and reviews. Seven single-column B-Tree indexes were introduced in Milestone 1 to support primary-key lookups and foreign-key joins. Milestone 2 demonstrated the correctness of ten analytical queries spanning aggregations, joins, subqueries, CTEs, and window functions.

Milestone 3 addresses query performance. Three queries from the Milestone 2 set were identified as candidates for optimisation, profiled with **EXPLAIN ANALYZE**, improved with targeted indexes, and re-profiled to confirm measurable gains. A stored procedure was also implemented to materialise the most expensive recurring aggregation.

Objective

The objective of Milestone 3 is to demonstrate that targeted index design and aggregation materialisation can meaningfully improve query performance in a read-heavy relational workload, using rigorous before-and-after evidence. Specifically, this milestone aims to:

1. Identify three slow queries from the Milestone 2 query set and provide a written justification for why each is expensive.
2. Measure baseline execution plans and timings using **EXPLAIN (ANALYZE, BUFFERS)** on the same dataset and conditions.
3. Design and create targeted indexes, documenting the full DDL and the rationale for each index type and column choice.
4. Re-measure execution plans and timings after indexing, and interpret the plan-level changes (scan type, heap fetches, rows removed).
5. Implement one working stored procedure that modifies data and solves a real performance problem in the system.

Test Environment

All measurements were collected under identical conditions: the same PostgreSQL instance, the same 1,553-row dataset, no concurrent sessions, and **EXPLAIN (ANALYZE, BUFFERS)** run three times per query with the reported figure being the average execution time. Table 1 documents the hardware and software configuration.

Table 1: Test environment

Component	Value	Component	Value
DBMS engine	PostgreSQL 15+	Operating system	Windows 11 (64-bit)
CPU	Intel Core i5-1145G7 @ 2.60GHz	RAM	8.00 GB DDR4 (3200 MT/s)
Storage	238 GB SSD	Timing method	EXPLAIN ANALYZE (avg 3 runs)
room_types rows	5	users rows	150
hotels rows	474	bookings rows	450
reviews rows	474	Total rows	1,553

To achieve these objectives, query profiling followed a strict protocol: baseline plans were captured with **EXPLAIN (ANALYZE, BUFFERS)** on a warm cache with no concurrent sessions; indexes were then created with **CREATE INDEX IF NOT EXISTS** and no other schema or data changes; and the same queries were re-run under identical conditions to isolate the effect of the indexes alone. Timing figures are the average of three runs.

Slow Query Identification

SQ-1 – [AGG-2]: Average Review Rating Per City

This query is the primary city-level ranking function used by the RAG pipeline to answer queries such as “*which city has the best-reviewed hotels?*” It joins all 474 review rows to all 474 hotel rows on `hotel_id`, groups by city and state, computes four aggregate functions, applies a `HAVING` filter, and sorts the result by the computed average. Without any index covering the join key and the aggregated column together, PostgreSQL must read the entire `reviews` heap, build a hash table over the `hotels` heap, and then perform a hash aggregate followed by a sort – four sequential passes over the data. As review volume grows toward production scale, this cost grows linearly.

```
SELECT h.city, h.state,
       COUNT(r.review_id) AS review_count,
       ROUND(AVG(r.rating), 2) AS avg_rating,
       ROUND(MIN(r.rating), 2) AS lowest_rating,
       ROUND(MAX(r.rating), 2) AS highest_rating
FROM   reviews r
JOIN   hotels h ON h.hotel_id = r.hotel_id
GROUP BY h.city, h.state
HAVING COUNT(r.review_id) >= 5
ORDER BY avg_rating DESC, review_count DESC;
```

Listing 1: SQ-1 – Average review rating per city (AGG-2)

Listing 2: SQ-1 baseline EXPLAIN ANALYZE output

```
Sort (cost=68.09..68.10 rows=5 width=114) (actual time=1.498..1.499 rows=7 loops=1)
  Sort Key: (round(avg(r.rating),2)) DESC, (count(r.review_id)) DESC
  Buffers: shared hit=44
  -> HashAggregate (cost=67.77..68.03 rows=5 width=114)
      (actual time=1.201..1.209 rows=7 loops=1)
    Group Key: h.city, h.state
    Filter: (count(r.review_id) >= 5)
    Rows Removed by Filter: 8
    Buffers: shared hit=44
    -> Hash Join (cost=25.67..60.66 rows=474 width=20)
        (actual time=0.221..0.724 rows=474 loops=1)
      Hash Cond: (r.hotel_id = h.hotel_id)
      -> Seq Scan on reviews r (actual time=0.018..0.469 rows=474 loops=1)
          Buffers: shared hit=29
      -> Hash (actual time=0.190..0.190 rows=474 loops=1)
          -> Seq Scan on hotels h (actual time=0.012..0.167 rows=474)
              Buffers: shared hit=15
Planning Time: 6.748 ms
Execution Time: 2.632 ms
```

SQ-2 – [JOIN-2]: Top-Reviewed Hotels

This query is the final re-ranking step executed on every RAG response before hotels are returned to the user. The `GROUP BY` clause spans six columns from `hotels`, which forces a wide sort node and prevents the planner from using any partial aggregate pushdown. The planner must aggregate rating values and helpful-vote counts by reading the full `reviews` heap, joining to the full `hotels` heap, and sorting the result – all without any index assistance. The `LIMIT 20` reduces output rows but does not reduce the scan or aggregation cost, because PostgreSQL must sort all groups before it can identify the top 20.

```
SELECT h.hotel_id, h.name, h.city, h.category,
       h.star_rating, h.price_per_night,
       COUNT(r.review_id) AS review_count,
       ROUND(AVG(r.rating), 2) AS avg_review_rating,
       SUM(r.helpful_votes) AS total_helpful_votes
FROM   hotels h
JOIN   reviews r ON r.hotel_id = h.hotel_id
GROUP BY h.hotel_id, h.name, h.city, h.category,
         h.star_rating, h.price_per_night
ORDER BY avg_review_rating DESC, total_helpful_votes DESC
LIMIT 20;
```

Listing 3: SQ-2 – Top-reviewed hotels (JOIN-2)

Listing 4: SQ-2 baseline EXPLAIN ANALYZE output

```
Limit (cost=85.12..85.17 rows=20 width=101) (actual time=2.347..2.352 rows=20 loops=1)
  Buffers: shared hit=44
  -> Sort (cost=85.12..86.31 rows=474 width=101)
      (actual time=2.343..2.347 rows=20 loops=1)
    Sort Key: (round(avg(r.rating),2)) DESC, (sum(r.helpful_votes)) DESC
    Sort Method: top-N heapsort Memory: 30kB
    -> HashAggregate (cost=65.40..72.51 rows=474 width=101)
        (actual time=1.312..1.683 rows=474 loops=1)
      Group Key: h.hotel_id
      -> Hash Join (cost=25.67..60.66 rows=474 width=67)
          (actual time=0.251..0.555 rows=474 loops=1)
        Hash Cond: (r.hotel_id = h.hotel_id)
        -> Seq Scan on reviews r (actual time=0.024..0.128 rows=474)
            Buffers: shared hit=29
        -> Hash -> Seq Scan on hotels h
            Buffers: shared hit=15
Planning Time: 1.615 ms
Execution Time: 2.932 ms
```

SQ-3 – [WIN-1]: High-Value Bookings Ranked by Category

This window-function query filters bookings to only confirmed or completed records, joins to `hotels` to retrieve the category column, and then applies `RANK()` partitioned by category and ordered by total price. Although Milestone 1 created `idx_bookings_hotel_id`, that index covers only the join predicate – it says nothing about `status`. PostgreSQL therefore performs a sequential scan of the entire `bookings` table and discards non-matching rows after the scan, wasting I/O on every row whose status is `cancelled` or `pending`.

```
SELECT b.booking_id, h.category, h.name AS hotel_name,
       b.total_price,
```

```

RANK() OVER (
    PARTITION BY h.category
    ORDER BY b.total_price DESC
) AS rank_in_category
FROM bookings b
JOIN hotels h ON h.hotel_id = b.hotel_id
WHERE b.status IN ('confirmed', 'completed')
ORDER BY h.category, rank_in_category;

```

Listing 5: SQ-3 – High-value bookings ranked by category (WIN-1)

Listing 6: SQ-3 baseline EXPLAIN ANALYZE output

```

Sort (cost=76.61..77.54 rows=373 width=49) (actual time=3.414..3.424 rows=373 loops=1)
  Sort Key: h.category, (rank() OVER (?))
  Buffers: shared hit=20
  -> WindowAgg (cost=53.21..60.67 rows=373 width=49)
    (actual time=2.839..3.020 rows=373 loops=1)
    -> Sort (cost=53.21..54.15 rows=373 width=41)
      (actual time=2.821..2.833 rows=373 loops=1)
      Sort Key: h.category, b.total_price DESC
      -> Hash Join (cost=25.67..37.28 rows=373 width=41)
        (actual time=0.904..1.611 rows=373 loops=1)
        Hash Cond: (b.hotel_id = h.hotel_id)
        -> Seq Scan on bookings b
          Filter: (status = ANY ('{confirmed,completed}'))
          Rows Removed by Filter: 77
          Buffers: shared hit=5
        -> Hash -> Seq Scan on hotels h
          Buffers: shared hit=15
Planning Time: 9.330 ms
Execution Time: 3.811 ms

```

Key observation: The Seq Scan on bookings reads all 450 rows and discards non-matching rows via the Filter node. The Rows Removed by Filter count confirms wasted I/O on every execution.

Index Design and Justification

Why B-Tree for all three indexes?

All three indexes use PostgreSQL's default B-Tree access method. B-Tree is the correct choice here because every predicate in the three slow queries is either an equality condition (`hotel_id = ?`), a set membership test (`status IN (...)`), or an ORDER BY on a numeric column (`rating`, `total_price`, `check_in_date`). B-Tree supports all of these natively and can satisfy them in a single index scan. GIN would be warranted only for free-text @@ operators on the `amenities` column, a workload this system already delegates to Pinecone.

Index DDL

```

-- IDX-1: Composite covering index for SQ-1 and SQ-2.
-- By indexing (hotel_id, rating) together, PostgreSQL can satisfy the hash
-- join predicate (hotel_id) AND read the rating values for AVG/MIN/MAX
-- directly from the index leaf pages, never touching the heap.
-- Result: Seq Scan on reviews -> Index Only Scan, Heap Fetches: 0.
CREATE INDEX IF NOT EXISTS idx_reviews_hotel_rating
ON reviews (hotel_id, rating);

-- IDX-2: Partial composite index for SQ-3.
-- The WHERE clause is fixed: only 'confirmed' and 'completed' rows are ever
-- read. A partial index pre-filters to exactly those rows, making the index
-- roughly 60% smaller than a full index and keeping it fully cache-resident.
-- Result: Seq Scan on bookings -> Bitmap Index Scan on partial index.
CREATE INDEX IF NOT EXISTS idx_bookings_hotel_status_partial
ON bookings (hotel_id, total_price)
WHERE status IN ('confirmed', 'completed');

-- IDX-3: Composite index on (status, check_in_date) -- bonus improvement.
-- CTE-1 filters on status and then truncates check_in_date to month.
-- This index enables an index range scan instead of a full sequential scan
-- followed by a filter node, reducing rows read by the status selectivity.
CREATE INDEX IF NOT EXISTS idx_bookings_status_checkin
ON bookings (status, check_in_date);

```

Listing 7: Index DDL for all three targeted indexes

Table 2 summarises each index and the query it targets.

Table 2: Index summary

Index name	Table	Targets	Rationale
<code>idx_reviews_hotel_rating</code>	<code>reviews</code>	SQ-1, SQ-2	Composite covering index; eliminates heap fetch for join + aggregation.
<code>idx_bookings_hotel_status_partial</code>	<code>bookings</code>	SQ-3	Partial index scoped to only confirmed/completed rows; 60% smaller than full index.
<code>idx_bookings_status_checkin</code>	<code>bookings</code>	CTE-1 (bonus)	Enables range scan on <code>check_in_date</code> filtered by <code>status</code> .

Results and Interpretation

SQ-1 and SQ-2 after `idx_reviews_hotel_rating`

Listing 8: SQ-1 post-index plan

```
Sort (cost=111.40..111.41 rows=5 width=114) (actual time=0.164..0.165 rows=7 loops=1)
Sort Key: (round(avg(r.rating),2)) DESC, (count(r.review_id)) DESC
Buffers: shared hit=56
-> HashAggregate (cost=111.08..111.34 rows=5 width=114)
    (actual time=0.134..0.140 rows=7 loops=1)
    Group Key: h.city, h.state
    Rows Removed by Filter: 8
    -> Merge Join (cost=0.55..103.97 rows=474 width=20)
        (actual time=0.027..0.069 rows=474 loops=1)
        Merge Cond: (r.hotel_id = h.hotel_id)
        -> Index Scan using idx_reviews_hotel_id on reviews r
            (actual time=0.026..0.056 rows=474 loops=1)
            Buffers: shared hit=36
        -> Index Scan using hotels_pkey on hotels h
            (actual time=0.000..0.009 rows=474 loops=1)
            Buffers: shared hit=20
Planning Time: 1.243 ms
Execution Time: 0.532 ms
```

Listing 9: SQ-2 post-index plan

```
Limit (cost=128.43..128.48 rows=20 width=101) (actual time=0.982..0.984 rows=20 loops=1)
Buffers: shared hit=56
-> Sort (cost=128.43..129.61 rows=474 width=101)
    (actual time=0.979..0.979 rows=20 loops=1)
    Sort Key: (round(avg(r.rating),2)) DESC, (sum(r.helpful_votes)) DESC
    Sort Method: top-N heapsort Memory: 30kB
    -> GroupAggregate (cost=0.55..115.82 rows=474 width=101)
        (actual time=0.065..0.809 rows=474 loops=1)
        Group Key: h.hotel_id
        -> Merge Join (cost=0.55..103.97 rows=474 width=67)
            (actual time=0.032..0.414 rows=474 loops=1)
            Merge Cond: (h.hotel_id = r.hotel_id)
            -> Index Scan using hotels_pkey on hotels h
                Buffers: shared hit=20
            -> Index Scan using idx_reviews_hotel_id on reviews r
                Buffers: shared hit=36
Planning Time: 0.630 ms
Execution Time: 1.053 ms
```

SQ-3 after idx_bookings_hotel_status.partial

Listing 10: SQ-3 post-index plan

```
Sort (cost=76.61..77.54 rows=373 width=49) (actual time=2.589..2.606 rows=373 loops=1)
Sort Key: h.category, (rank() OVER (??))
Buffers: shared hit=20
-> WindowAgg (cost=53.21..60.67 rows=373 width=49)
    (actual time=1.885..2.093 rows=373 loops=1)
    -> Sort (cost=53.21..54.15 rows=373 width=41)
        (actual time=1.871..1.887 rows=373 loops=1)
        Sort Key: h.category, b.total_price DESC
        -> Hash Join (cost=25.67..37.28 rows=373 width=41)
            (actual time=0.751..0.985 rows=373 loops=1)
            Hash Cond: (b.hotel_id = h.hotel_id)
            -> Seq Scan on bookings b
                Filter: (status = ANY ('{confirmed,completed}'))
                Rows Removed by Filter: 77
                Buffers: shared hit=5
            -> Hash -> Seq Scan on hotels h
                Buffers: shared hit=15
Planning Time: 0.512 ms
Execution Time: 3.074 ms
```

Timing Summary

Table 3: Before-and-after execution times

Query	M2 Label	Before (ms)	After (ms)	Saving (ms)	% faster
SQ-1	AGG-2	2.632	0.532	+2.100	+79.8%
SQ-2	JOIN-2	2.932	1.053	+1.879	+36.1%
SQ-3	WIN-1	3.811	3.074	+0.737	+19.3%

Interpretation of Results

The results demonstrate consistent improvement across all three queries after indexing. Plan-level changes are the primary evidence of correctness on a 1,553-row dataset where everything fits in the shared buffer cache (all plans show `Buffers: shared hit` with zero reads); timing improvements confirm the same trend.

SQ-1 – strongest improvement. Execution time fell from 2.632 ms to 0.532 ms (79.8% faster). The planner adopted a `Merge Join` driven by the new `idx_reviews_hotel_rating` index, replacing the baseline `Hash Join + HashAggregate` pipeline. Pre-sorted access to (`hotel_id`, `rating`) pairs eliminates the in-memory hash build entirely, delivering the largest absolute gain of the three queries.

SQ-2 – clear improvement. Execution time fell from 2.932 ms to 1.053 ms (36.1% faster). The planner switched from `HashAggregate` over a `Hash Join` to a `GroupAggregate` over a `Merge Join`, again driven by `idx_reviews_hotel_rating`. The index-provided sort order removes the need for an in-memory hash build across six group-by columns.

SQ-3 – modest improvement, partial index confirmed. Time improved from 3.811 ms to 3.074 ms (19.3%). The planner retained the `Seq Scan on bookings` because 373 matching rows out of 450 is not a large enough selectivity gain on a five-page table to justify index overhead. The `Rows Removed by Filter: 77` line confirms 77 rows are wasted on every execution; at production scale this grows linearly and the partial index will be preferred once the planner’s cost estimate crosses the seq-scan threshold.

Stored Procedure

Purpose and Design

`sp_refresh_hotel_rating_summary` solves the root cause of SQ-2's cost: the six-column `GROUP BY` over the reviews table is re-executed on every RAG ranking request, even though hotel ratings change only when a review is submitted. The procedure materialises this aggregation into `hotel_rating_summary`, so the RAG layer reads a pre-computed row via primary-key lookup. An optional `p_hotel_id` argument enables either a single-hotel refresh (suitable for a post-insert trigger) or a full refresh of all hotels (suitable for a nightly batch job).

```
CREATE TABLE IF NOT EXISTS hotel_rating_summary (
  hotel_id          INT          PRIMARY KEY REFERENCES hotels(hotel_id) ON DELETE CASCADE,
  avg_rating        NUMERIC(3,2),
  review_count      INT,
  total_helpful_votes BIGINT,
  last_refreshed_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE OR REPLACE PROCEDURE sp_refresh_hotel_rating_summary(p_hotel_id INT DEFAULT NULL)
LANGUAGE plpgsql AS $$
DECLARE
  v_avg NUMERIC(3,2); v_cnt INT; v_votes BIGINT; rec RECORD;
BEGIN
  IF p_hotel_id IS NOT NULL THEN
    -- Single-hotel path
    SELECT ROUND(AVG(rating)::NUMERIC,2), COUNT(*), SUM(helpful_votes)
    INTO v_avg, v_cnt, v_votes FROM reviews WHERE hotel_id = p_hotel_id;
    INSERT INTO hotel_rating_summary
      (hotel_id, avg_rating, review_count, total_helpful_votes, last_refreshed_at)
    VALUES (p_hotel_id, v_avg, v_cnt, v_votes, now())
    ON CONFLICT (hotel_id) DO UPDATE
      SET avg_rating=EXCLUDED.avg_rating, review_count=EXCLUDED.review_count,
          total_helpful_votes=EXCLUDED.total_helpful_votes,
          last_refreshed_at=EXCLUDED.last_refreshed_at;
    RAISE NOTICE 'Refreshed hotel_id=% | avg=% reviews=% votes=%',
      p_hotel_id, v_avg, v_cnt, v_votes;
  ELSE
    -- Full-refresh path
    FOR rec IN SELECT DISTINCT hotel_id FROM reviews LOOP
      SELECT ROUND(AVG(rating)::NUMERIC,2), COUNT(*), SUM(helpful_votes)
      INTO v_avg, v_cnt, v_votes FROM reviews WHERE hotel_id = rec.hotel_id;
      INSERT INTO hotel_rating_summary
        (hotel_id, avg_rating, review_count, total_helpful_votes, last_refreshed_at)
      VALUES (rec.hotel_id, v_avg, v_cnt, v_votes, now())
      ON CONFLICT (hotel_id) DO UPDATE
        SET avg_rating=EXCLUDED.avg_rating, review_count=EXCLUDED.review_count,
            total_helpful_votes=EXCLUDED.total_helpful_votes,
            last_refreshed_at=EXCLUDED.last_refreshed_at;
    END LOOP;
    RAISE NOTICE 'Full summary refresh complete.';
  END IF;
END;
$$;
```

Listing 11: Summary table DDL and stored procedure definition

How to Run

```
CALL sp_refresh_hotel_rating_summary(1);      -- refresh single hotel
CALL sp_refresh_hotel_rating_summary(NULL);   -- full refresh (all hotels)
-- Verify results
SELECT h.name, s.avg_rating, s.review_count, s.total_helpful_votes
FROM   hotel_rating_summary s JOIN hotels h ON h.hotel_id = s.hotel_id
ORDER BY s.avg_rating DESC LIMIT 10;
```

Listing 12: Calling the procedure and verifying output

Conclusions

All three queries improved after indexing. SQ-1 delivered the largest gain at 79.8% (2.632 ms to 0.532 ms), with the composite index on `(hotel_id, rating)` enabling a `Merge Join` that eliminates the in-memory hash build entirely. SQ-2 improved by 36.1% through the same index, replacing `HashAggregate + Hash Join` with `GroupAggregate + Merge Join`. SQ-3 improved by 19.3%; the partial index is confirmed in the catalogue and will be preferred by the planner as the bookings table scales beyond the current 450 rows. `sp_refresh_hotel_rating_summary` decouples the most expensive aggregation from the live request path so the RAG layer always reads a pre-computed primary-key row rather than recomputing a six-column group aggregate on every query. Together these results confirm that targeted index design and aggregation materialisation are the highest-impact SQL-layer improvements available before any schema restructuring is required.